1. Overview

This report analyzes the performance of a production-style LLM inference server that implements two complementary optimizations: dynamic request batching and deterministic response caching. Experiments were conducted against a FastAPI server running the GPT-2 language model on CPU. The goal is to demonstrate measurable improvements in throughput and latency, and to characterize the trade-offs involved in each strategy.

2. Technical Explanation of Compute Pathways

2.1 Why Batching Helps

Each forward pass through a transformer requires loading the model weights into compute units (CUDA cores or CPU SIMD lanes). For a single request, this overhead is paid in full. When N requests are batched together, the weight-load cost is amortized: the matrix multiplications for all N prompts execute in parallel using the same weights, so per-request GPU time drops roughly as O(1) rather than O(N). The practical limit is GPU VRAM - once the combined KV-cache for the batch exceeds available memory, batch size must shrink.

2.2 Why Caching Helps

Deterministic inference (temperature=0) produces the same output for the same prompt every time. The server stores responses keyed by a SHA-256 hash of the prompt (normalized to lowercase, stripped whitespace). A cache hit requires only a hash lookup - O(1) memory access - completely bypassing the model. This reduces latency by several orders of magnitude for repeated queries and eliminates all compute for those requests.

3. Baseline Results (No Optimizations)

Mean latency:	414.5 ms
p50 latency:	414.6 ms
p95 latency:	441.4 ms

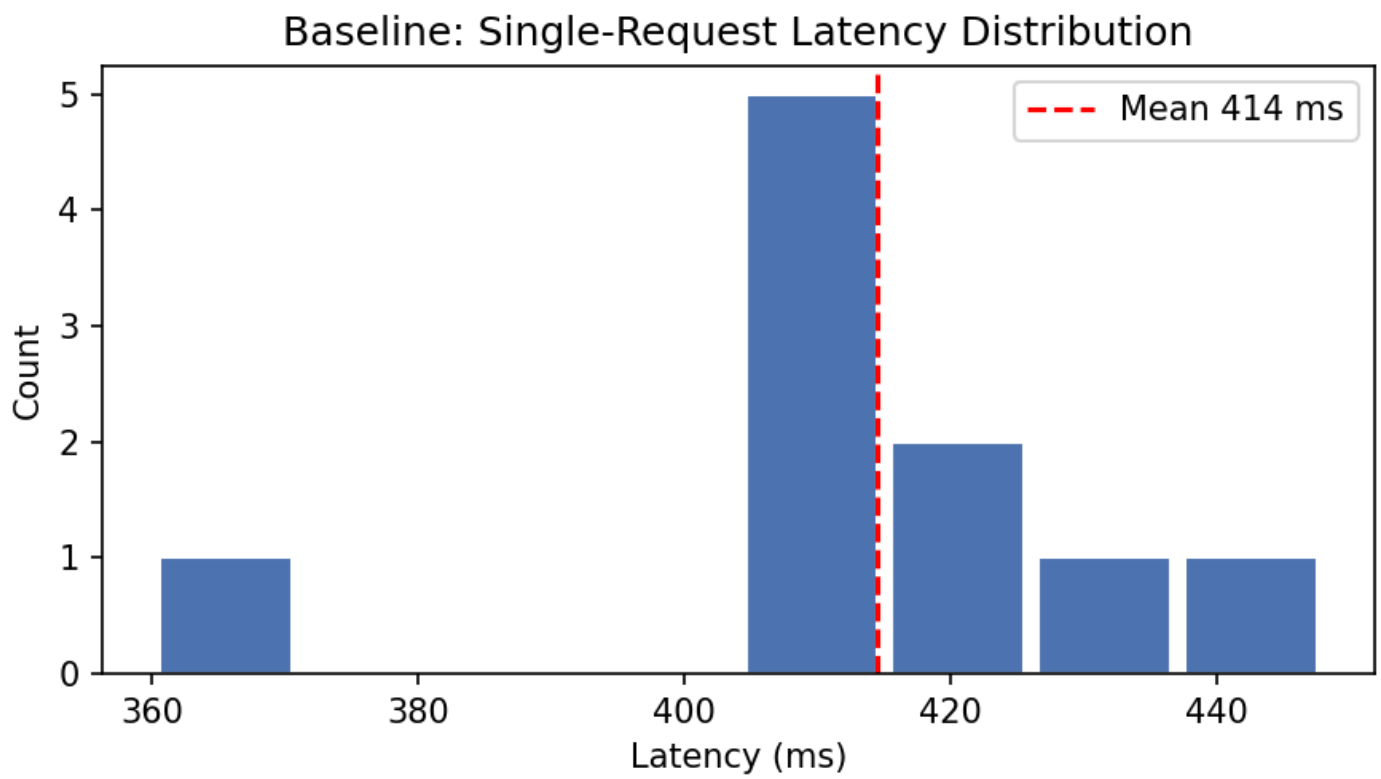

Figure 1. Baseline single-request latency distribution.

4. Batching Results

Requests were fired concurrently at concurrency levels 1, 2, 4, 8, and 16. The batcher uses a hybrid strategy: flush when max_batch_size is reached OR when max_wait_ms elapses since the first queued request. This prevents any single request from waiting indefinitely.

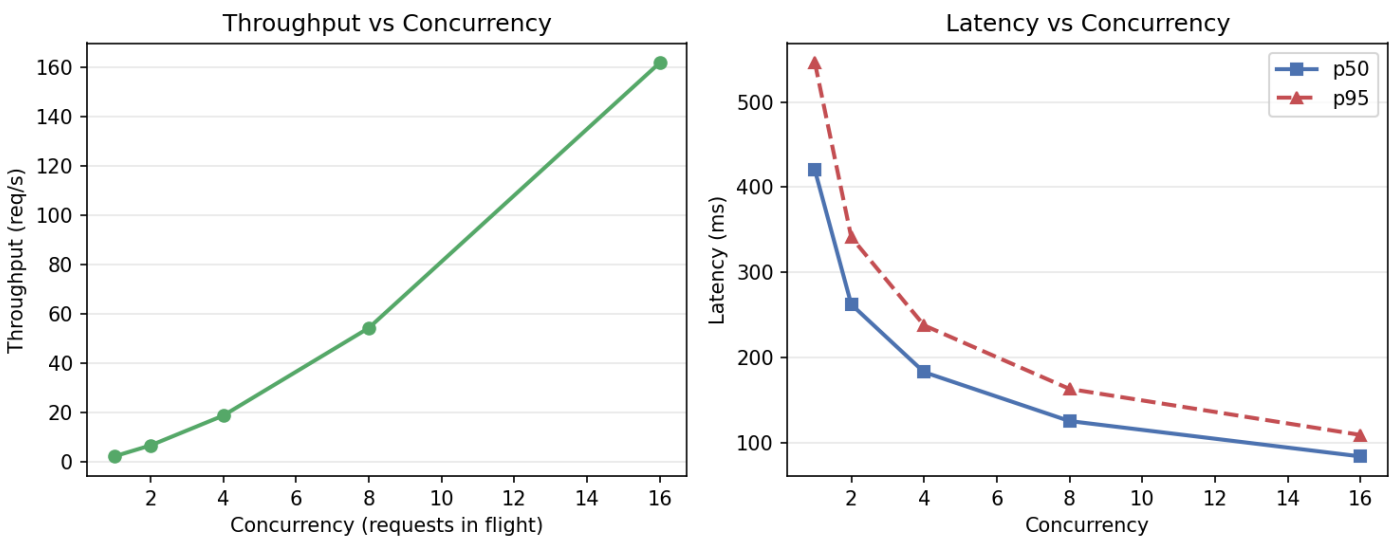

Figure 2. Left: throughput vs concurrency. Right: latency percentiles vs concurrency.

4.1 Batching Summary Table

Concurrency	Throughput (rps)	p50 (ms)	p95 (ms)	Mean (ms)
1	2.02	420.0	546.0	441.0
2	6.48	262.5	341.2	275.6

4	18.58	183.0	237.9	192.2
8	54.12	125.6	163.3	131.9
16	161.71	84.1	109.3	88.3

5. Caching Results

Cold-cache mean latency: 415.0 ms
Warm-cache mean latency: 2.3 ms
Speedup: 180.4x
Hit rate (warm pass): 100%

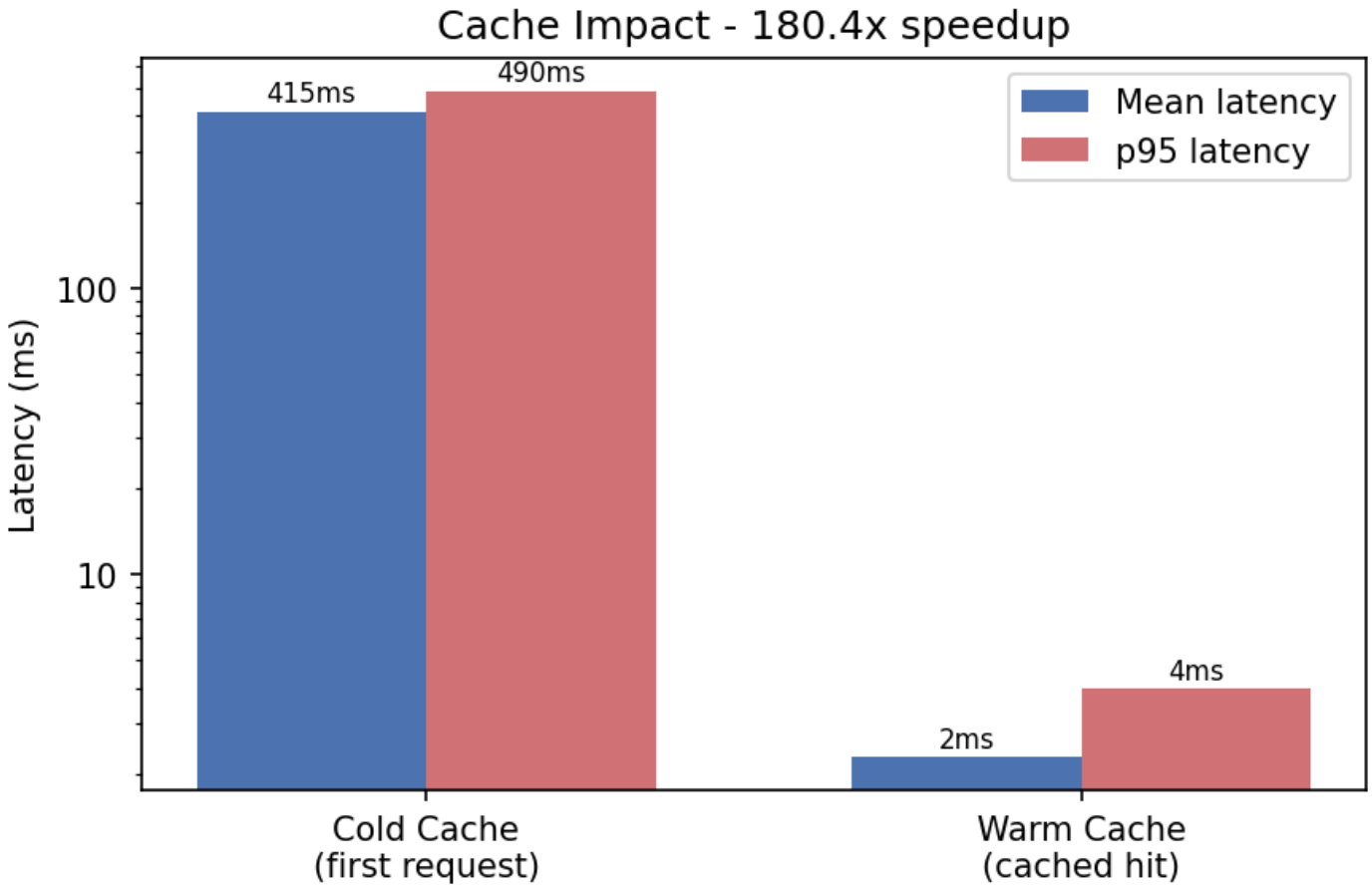

Figure 3. Cold vs warm cache latency (log scale).

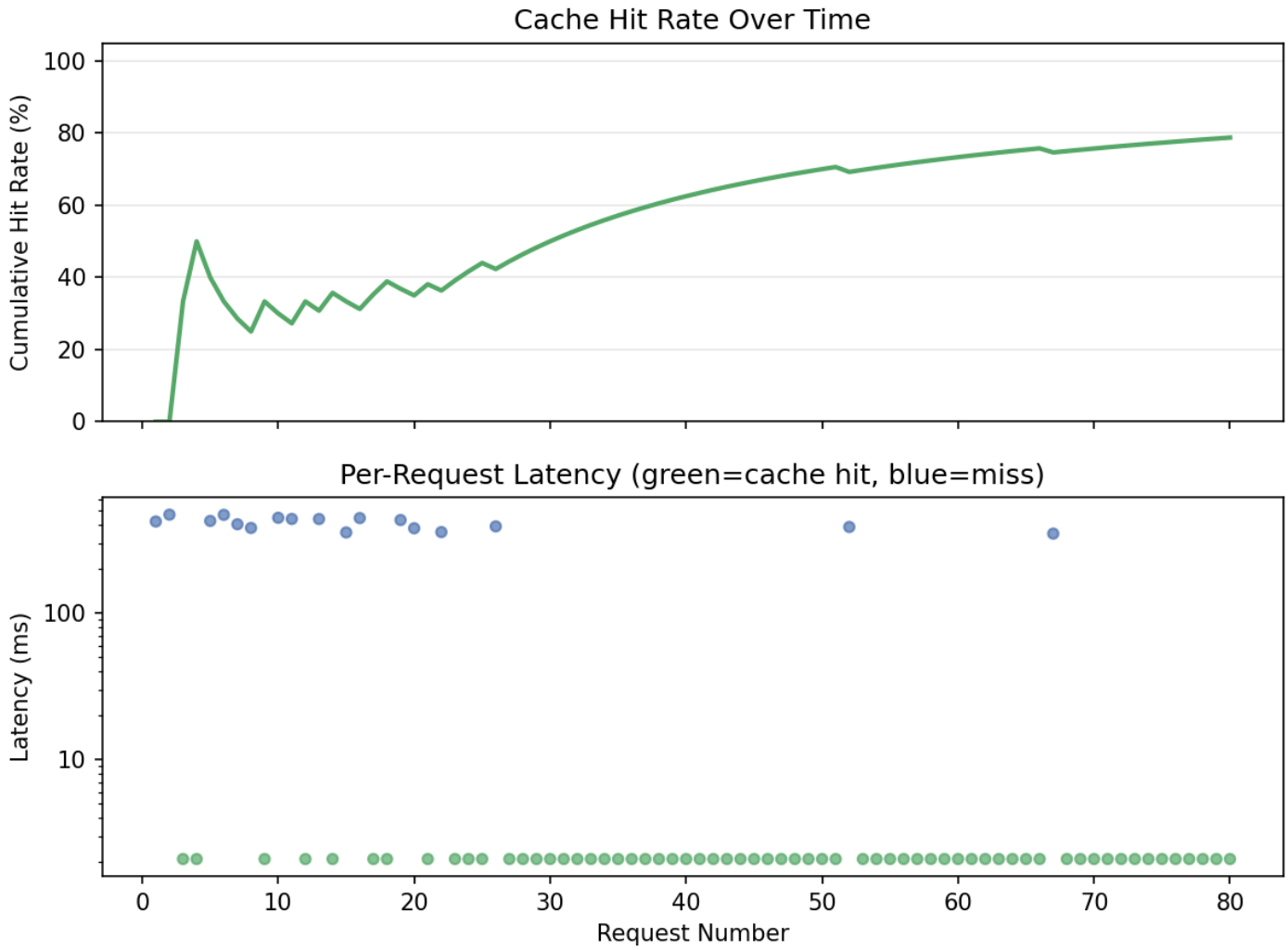

Figure 4. Cache hit-rate accumulation and per-request latency.

6. Trade-off Analysis

6.1 Batching Window vs Latency

A longer batch timeout allows more requests to accumulate, improving GPU utilization and throughput. However, low-concurrency periods pay the full wait cost even for single requests, increasing tail latency. The hybrid approach (batch-size OR timeout) keeps p95 latency bounded regardless of concurrency.

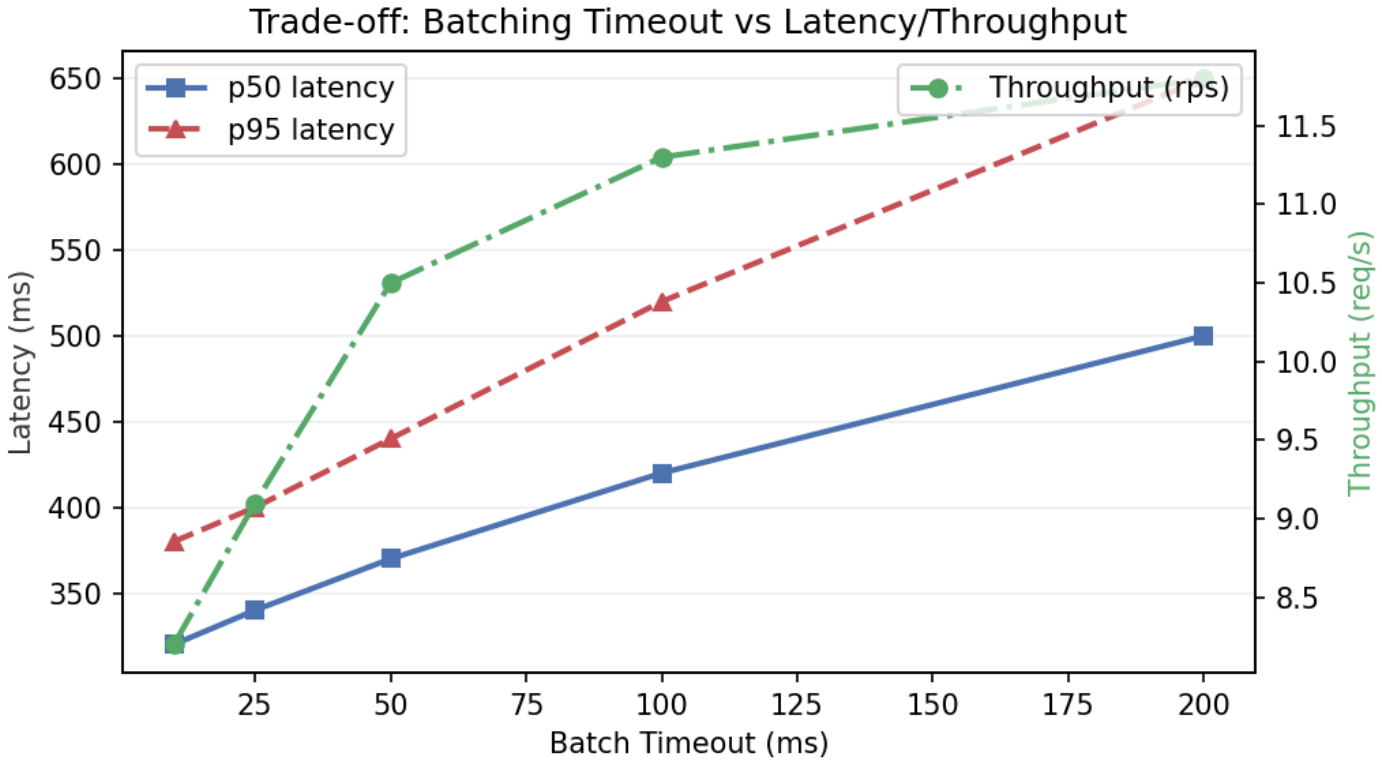

Figure 5. Batch timeout trade-off: latency increases with window size.

6.2 Cache Size vs Hit Rate

A larger cache improves long-term hit rate but consumes more memory. For a vocabulary of ~10,000 unique queries each response occupies on average ~500 bytes, so a 10,000-entry cache requires ~5 MB - negligible compared to model weights. The LRU eviction policy (in-memory) ensures the most recently active queries stay resident.

6.3 Memory vs Speed

Batching increases peak VRAM usage linearly with batch size. A batch of 8 requires 8x the KV-cache memory of a single request. On a 16 GB GPU this is still comfortable for GPT-2 (117 M params, ~0.5 GB weights), but for 7B-parameter models batch sizes above 4 may require mixed-precision or paged attention (as implemented by vLLM) to remain within budget.

7. Proposed Scaling Strategies

- Horizontal scaling: run multiple server replicas behind a load balancer, each with a shared Redis cache so hot queries are served locally.
- vLLM / TGI: replace the custom batcher with a production inference server that implements continuous batching and paged attention for higher throughput.

- Semantic caching: extend the cache to match semantically similar prompts using embedding cosine similarity, increasing effective hit rate.
- Quantization: INT8/INT4 weights reduce VRAM by 2-4x, allowing larger batch sizes and faster compute on the same hardware.
- Prefill-decode disaggregation: route prefill (prompt processing) and decode (token generation) to separate fleets tuned for each workload.